

WHATNEXT VISION MOTORS : SHAPING THE FUTURE OF MOBILITY WITH INNOVATION AND EXCELLENCE

COLLEGE:UNIVERSITY COLLEGE OF ENGINEERING
PATTUKOTTAI

COLLEGE CODE:8221

TEAM MEMBERS & REGISTER NO:

SANJAYA B(TEAM LEADER) - 822123104701
FATHIMA H - 822123104028
MUBHARAK C - 822123104043
ASHIK AHAMED A - 822123104702

EMAIL ID:

SANJAYA B - sanchayasanchaya200@gmail.com
FATHIMA H - abdullakwt449@gmail.com
MUBHARAK C - mubharakvip@gmail.com
ASHIK AHAMED A - ashikahamedsharp@gmail.com

NM ID :

SANJAYA B - F13111E6940067FA11C4849E0366B736
FATHIMA H - BB6346A52B18D2DB4CD440C2E50A939F
MUBHARAK C - 82442FB913CF1B2AC8747A5D9A43FFBB
ASHIK AHAMED A - CF1FB67C2B12C3EA4224B0631DAD2082

TEAM LEADER PROFILE LINK :

<https://www.salesforce.com/trailblazer/a4qx0vh6cyj3yiut0q>

APPLICATION LINK :

https://orgfarm-15fd1eb3de-dev-ed.develop.lightning.force.com/lightning/r/Vehicle_Order__c/a04jV000000WYszQAG/view

1.PROBLEM STATEMENT :

The existing vehicle sales process relies on manual management of customer orders, dealer allocation, and vehicle inventory, which often leads to delays, stock mismatches, and inefficient order processing. Customers may unknowingly place orders for unavailable vehicles, while manually assigning the nearest dealer increases processing time and reduces service efficiency. These challenges affect customer satisfaction and business productivity.

To address these issues, WhatsNext Vision Motors implements a Salesforce CRM solution that automates vehicle management, stock validation, dealer assignment, and order processing. The system ensures accurate inventory tracking, prevents orders for out-of-stock vehicles, and improves operational efficiency through automation.

2.OBJECTIVES :

- Develop a centralized Salesforce CRM system for vehicle sales management.
- Store and manage vehicle, dealer, customer, and order information efficiently.
- Automatically assign the nearest dealer based on customer location.
- Prevent vehicle orders when stock is unavailable.
- Automate order processing and status updates.
- Send automated reminders for scheduled test drives.
- Improve customer satisfaction through efficient and accurate services.
- Reduce manual effort using Salesforce automation tools.

3. CHALLENGES :

- Maintaining accurate vehicle stock availability.
- Automatically identifying the nearest dealer for customers.
- Preventing orders for out-of-stock vehicles.
- Managing customer orders efficiently.
- Automating business processes without errors.
- Ensuring timely notifications and order updates.
- Improving operational efficiency while reducing manual work.

4. SOLUTION APPROACH :

The proposed solution uses Salesforce CRM as a centralized platform to manage vehicles, customers, dealers, and orders. Custom Objects, Fields, and Relationships are used to organize business data efficiently.

Record-Triggered Flows automate dealer assignment and order processing, while Apex Triggers validate stock availability before confirming orders. Batch Apex periodically updates vehicle stock, and Scheduled Apex automates order status updates and email notifications. This solution improves customer experience, increases operational efficiency, and reduces manual effort.

5. MILESTONES :

Milestone 1:

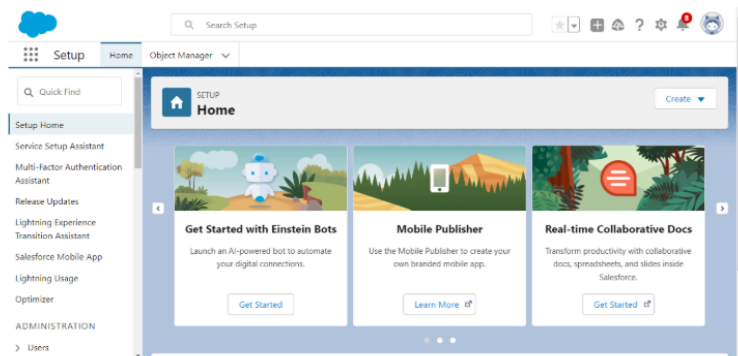
Activity 1: Creating Developer Account

Creating a developer org in salesforce.

1. Go to <https://developer.salesforce.com/signup>
2. First name & Last name
3. Email

4. Role : Developer
5. Company : College Name
6. County : India
7. Postal Code : pin code

Salesforce setup page :

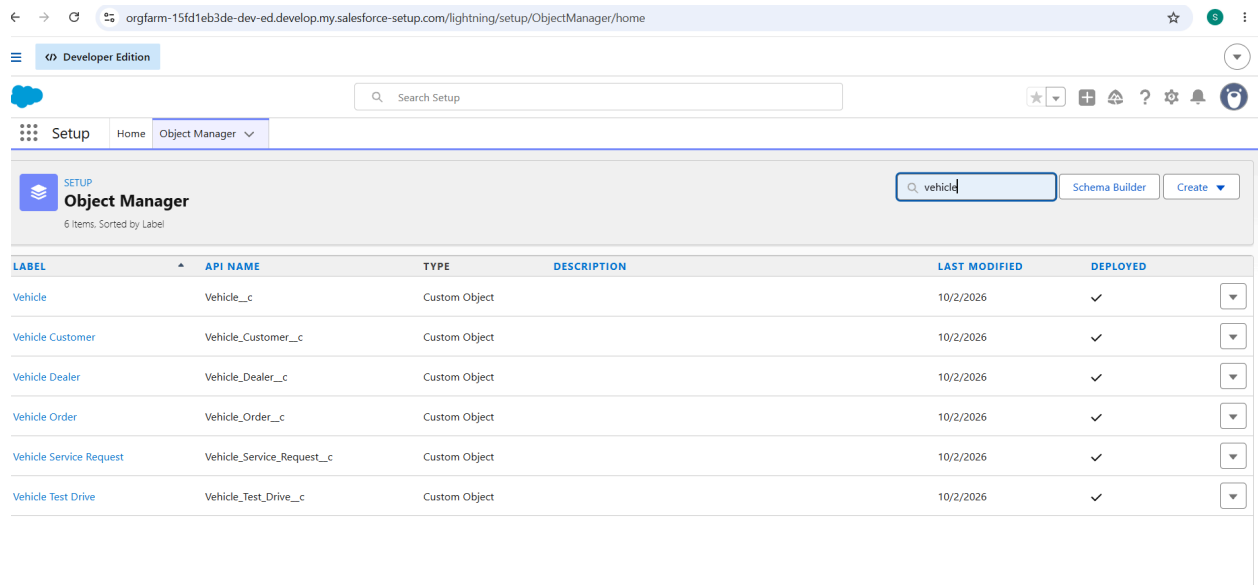


Milestone 2 : Create object

Objects & Relationships

Object Name	Purpose	Relationships
Vehicle__c	Stores vehicle details	Related to Dealer & Orders
Vehicle_Dealer__c	Stores authorized dealer info	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders & Test Drives

Vehicle_Order__c	Tracks vehicle purchases	Related to Customer & Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer & Vehicle
Vehicle_Service_Request__c	Tracks vehicle servicing requests	Related to Customer & Vehicle



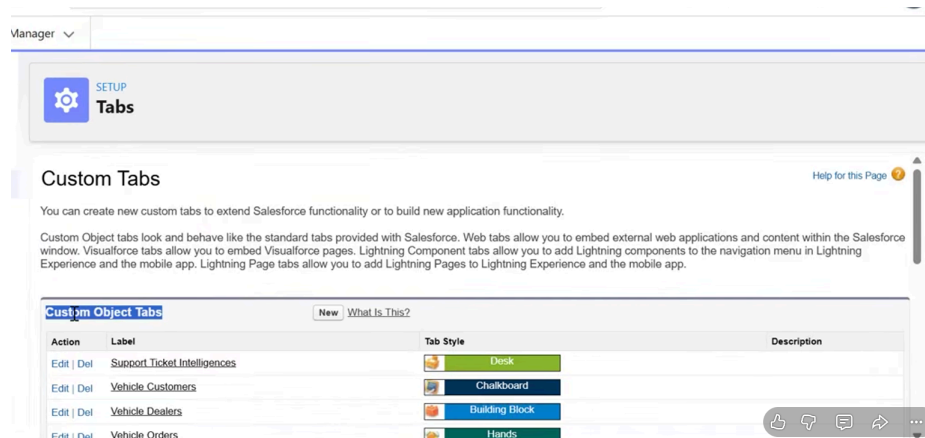
The screenshot shows the Salesforce Setup interface, specifically the Object Manager page. The breadcrumb trail is Setup > Home > Object Manager. A search bar at the top right contains the text 'vehicle'. Below the search bar, there are buttons for 'Schema Builder' and 'Create'. The main content area displays a table of custom objects with the following columns: LABEL, API NAME, TYPE, DESCRIPTION, LAST MODIFIED, and DEPLOYED. The table lists six objects, all of which are Custom Objects and were last modified on 10/2/2026. Each row has a dropdown arrow in the DEPLOYED column.

LABEL	API NAME	TYPE	DESCRIPTION	LAST MODIFIED	DEPLOYED
Vehicle	Vehicle__c	Custom Object		10/2/2026	✓
Vehicle Customer	Vehicle_Customer__c	Custom Object		10/2/2026	✓
Vehicle Dealer	Vehicle_Dealer__c	Custom Object		10/2/2026	✓
Vehicle Order	Vehicle_Order__c	Custom Object		10/2/2026	✓
Vehicle Service Request	Vehicle_Service_Request__c	Custom Object		10/2/2026	✓
Vehicle Test Drive	Vehicle_Test_Drive__c	Custom Object		10/2/2026	✓

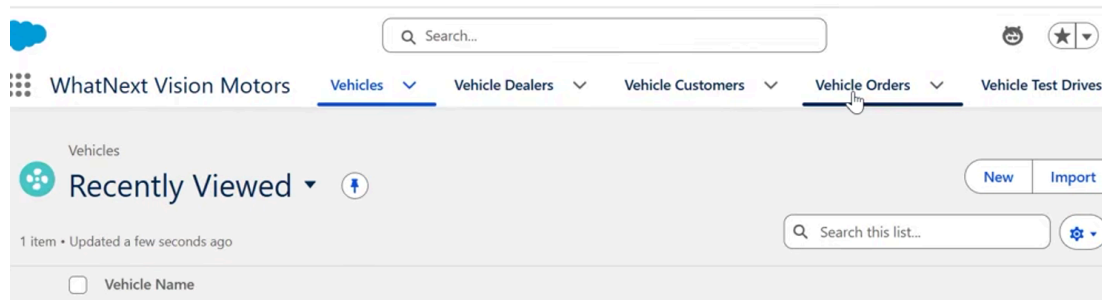
Milestone 3:

Activity 1: Creating a Custom Tab(Vehicle)

1. Go to setup page → type Tabs in Quick Find bar → click on tabs → New (under custom object tab)



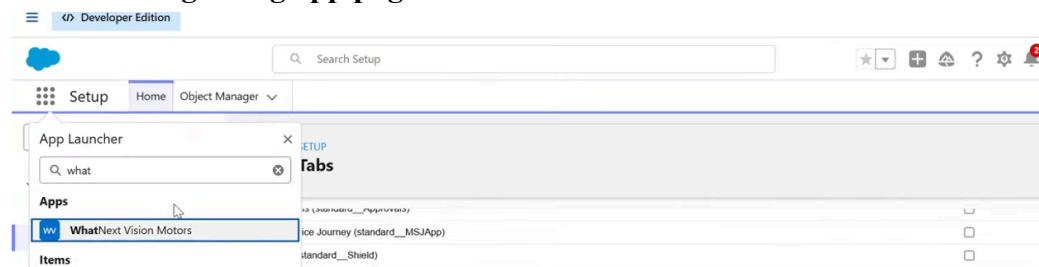
2. Activity 2:
3. Now create tabs for Other objects as well which we have created above Objects.



Milestone 4 - The Lightning App:

Activity 1: Create a Lightning App

To create a lightning app page:



App Name : [WhatNext Vision Motors](#)

Developer Name : [SANJAYA](#)

Description : [WhatNext Vision Motors](#) is a smart mobility platform that connects customers, vehicles, and services to make vehicle management simple, efficient, and future-ready.



Milestone 5 : Fields & Relationships

Key Fields for Each Object

1. Vehicle__c (Custom Object)

- **Vehicle Name**(Text)
- **Vehicle_Model__c** (Picklist: Sedan, SUV, EV, etc.)
- **Stock_Quantity__c** (Number)
- **Price__c** (Currency)
- **Dealer__c** (Lookup to Dealer__c)
- **Status__c** (Picklist: Available, Out of Stock, Discontinued)

2. Vehicle_Dealer__c (Custom Object)

- **Dealer_Name__c** (Text)
- **Dealer_Location__c** (Text)
- **Dealer_Code__c** (Auto Number)
- **Phone__c** (Phone)
- **Email__c** (Email)

3. Vehicle_Order__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Order_Date__c** (Date)
- **Status__c** (Picklist: Pending, Confirmed, Delivered, Canceled)

4. Vehicle_Customer__c (Custom Object)

- **Customer_Name__c** (Text)
- **Email__c** (Email)
- **Phone__c** (Phone)
- **Address__c** (Text)
- **Preferred_Vehicle_Type__c** (Picklist: Sedan, SUV, EV, etc.)

5. Vehicle_Test_Drive__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)

- **Test_Drive_Date__c** (Date)
- **Status__c** (Picklist: Scheduled, Completed, Canceled)

6. Vehicle_Service_Request__c (Custom Object)

- **Vehicle_Customer__c** (Lookup to Vehicle_Customer__c)
- **Vehicle__c** (Lookup to Vehicle__c)
- **Service_Date__c** (Date)
- **Issue_Description__c** (Text)
- **Status__c** (Picklist: Requested, In Progress, Completed)

The screenshot shows the Salesforce Setup interface for the 'Vehicle' custom object. The left sidebar contains navigation links: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, and Field Sets. The main content area is titled 'Vehicle' and 'Fields & Relationships', showing 10 items sorted by field label. A 'Quick Find' search bar is present. The table below lists the fields and their relationships.

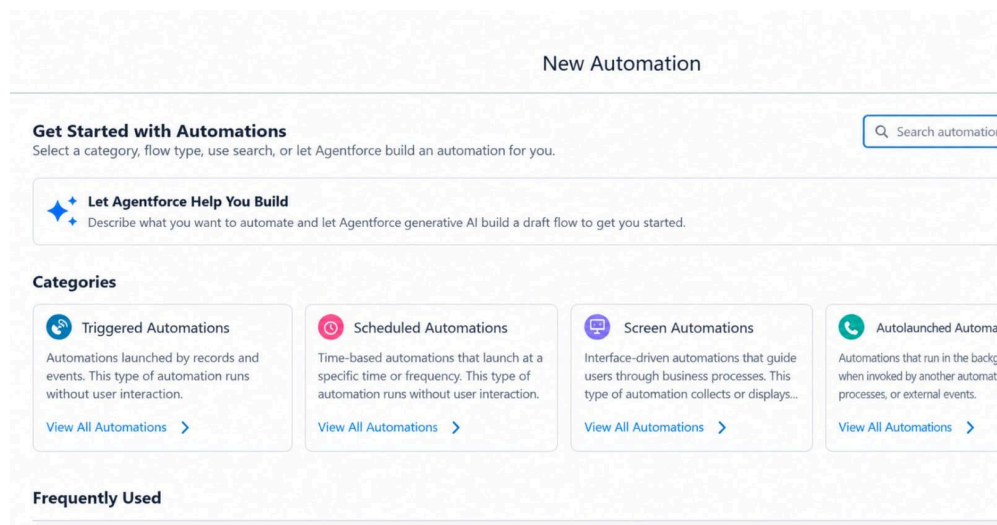
FIELD LABEL	FIELD NAME	DATA TYPE
Created By	CreatedById	Lookup(User)
Dealer	Dealer__c	Lookup(Vehicle Dealer)
Last Modified By	LastModifiedById	Lookup(User)
Owner	OwnerId	Lookup(User,Group)

MileStone 6 :

Flow 1 : Creating a record triggered flow to assign nearest dealer to the customer's location

Step 1 : In Quick Find, type Flows and click on Flows. Click New Flow.

Step 2 : Select Triggered Automation From Flow.



Step 4 : Select Vehicle Order Object

Trigger the Flow When : Select Record is Created.

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Pending

Step 5 : Click + → Select **Get Records**

Step 6 : Label : Get Customer Information

Object : Vehicle Customer

Condition :

Field : Id

Operator : Equals

Value : {!\$Record.Vehicle_Customer__c}

Step 7 : Click + → Select **Get Records**

Label : Get Nearest Dealer

Object : Vehicle Dealer

Condition :

Field : Dealer_Location__c

Operator : Equals

Value : {!Get_Customer_Information.Address__c}

Step 8 : Click + → Select **Update Records**

Label : Assign Dealer to Order

*How to Find Records to Update and Set Their Values : Use the IDs and all field values from a record or record collection

Select Record(s) to Update : {!Get_Nearest_Dealer}

Get Records of This Object

* Object

Vehicle Customer

Filter Vehicle Customer Records

Condition Requirements

All Conditions Are Met (AND)

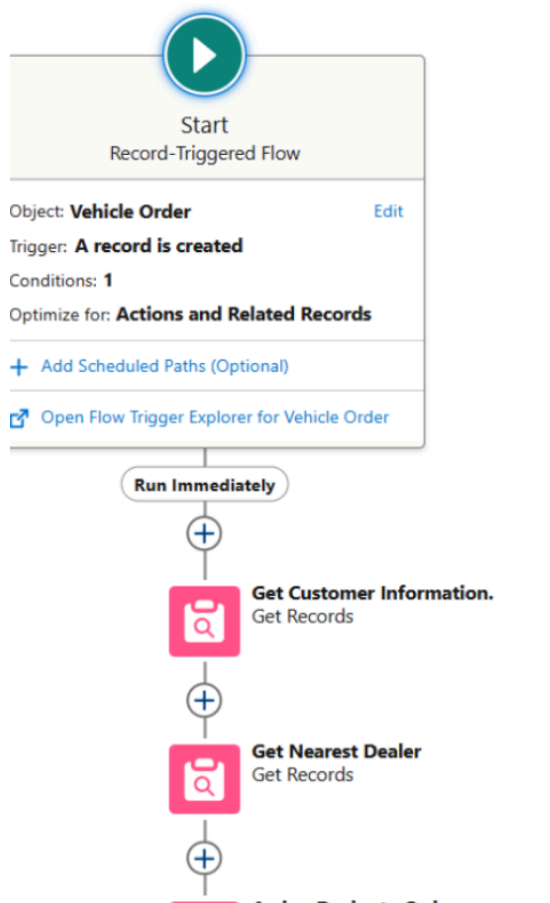
Records of This Object

Object

Vehicle Dealer

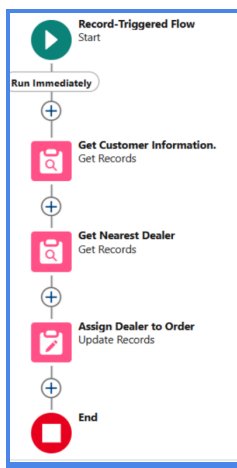
Filter Vehicle Dealer Records

Condition Requirements



Step 9 : Click Save and Give label Name and Activate Flow.

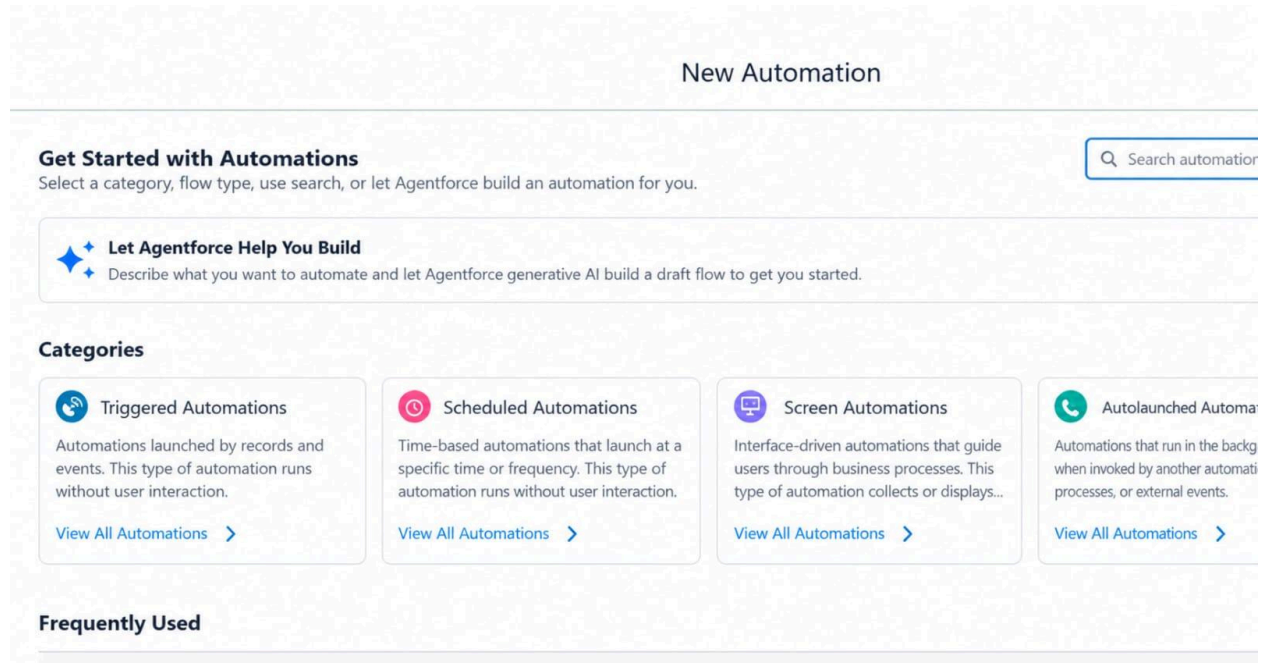
Label Name : Auto Assign Dealer



Step 10 : Activate Flow

Flow 2 : Creating record triggered flow to send an email to the customer reminding about the test drive.

Step 1 : Select Record-Triggered Flow



Step 2 : Select Vehicle Test Drive Object

Trigger the Flow When : A record is created or updated

Set Entry Condition :

All Conditions Are Met (AND)

Filed : Status__c

Operator : Equals

Value : Scheduled

Step 3 : Click + Add Scheduled Paths (below the trigger).

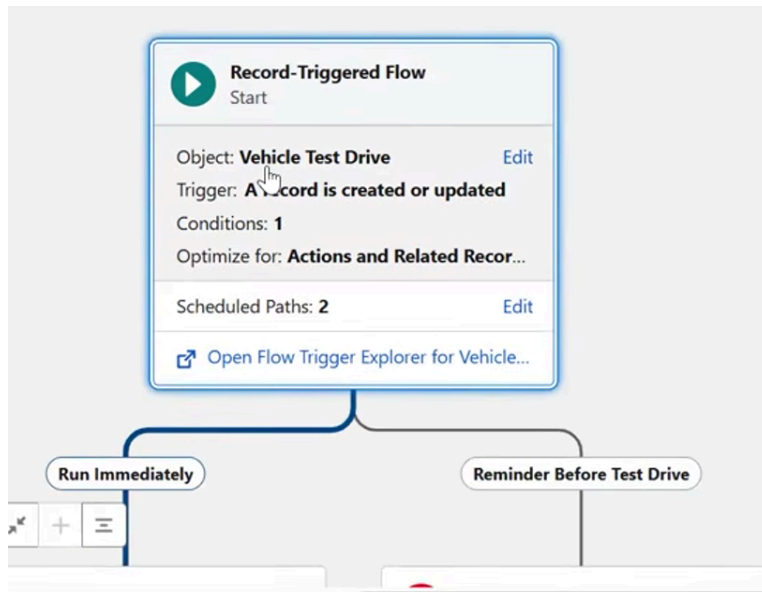
Label: Reminder Before Test Drive.

Time Source: Test_Drive_Date__c

Offset Number: 1.

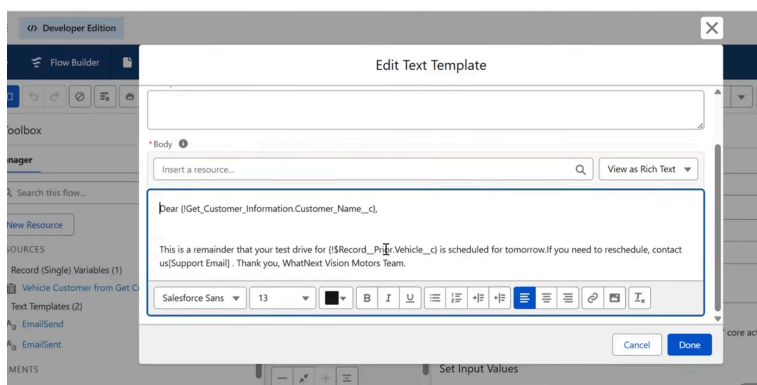
Offset Options: Days Before.

Click Done.



Step 4 :

1. Click + Add Element → Get Records.
2. Label: Get Customer Information.
3. Object: Vehicle_Customer__c.
4. Filter Conditions: Id = {!\$Record.Customer__c}.
5. How Many Records to Store: Select Only the first record.
6. How to Store Record Data: Choose Automatically store all fields.



Step 5: Send Reminder Email

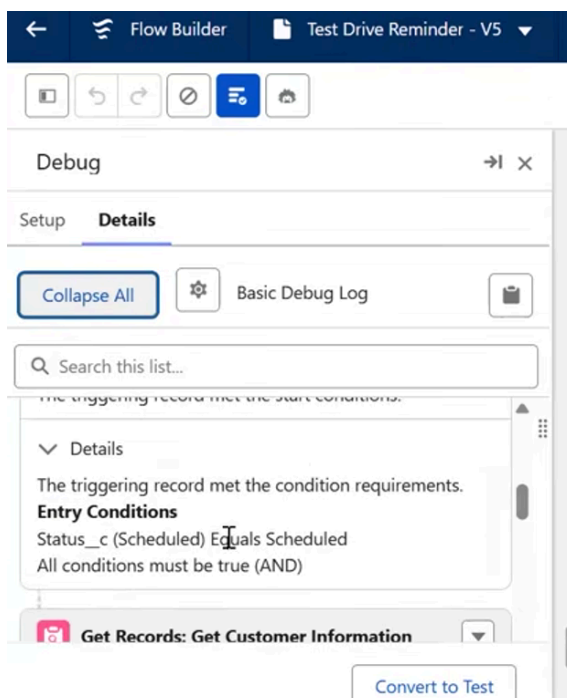
1. Click + Add Element → Action.
2. Action Type: Send Email
3. Label: Send Test Drive Reminder.
4. Subject: "Reminder: Your Test Drive is Tomorrow!".
5. Recipient Address List : {!Get_Customer_Information.Email__c}
6. Rich-Text-Formatted Body : True

Send Reminder Email:

Automatically sends a reminder email to customers before their scheduled test drive.

It ensures customers receive timely notifications.

Automatically sends a reminder email to customers before their drive




Send Email

A_a
Body ⓘ


EmailSent ✕

☒
Included

A_a
CC Recipient Address List

☐
Not Included

A_a
Email Template ID

☐
Not Included

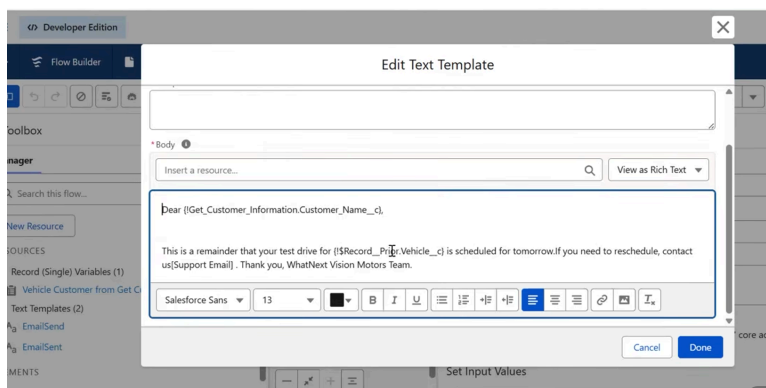
Log Email on Send

☐
Not Included

A_a
Recipient Address Collection

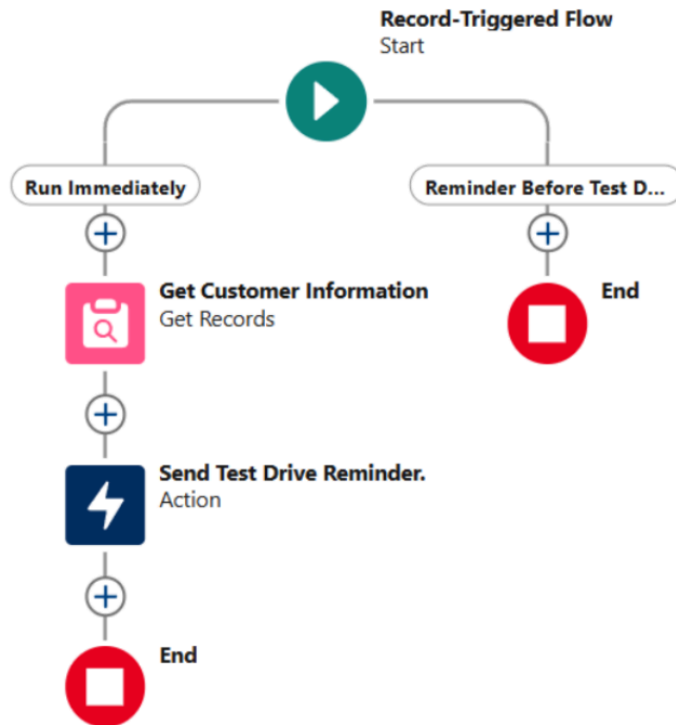
☐
Not Included

1. Body : Create Variable
2. Api Name : EmailSent



- 3.
4. Email flow creation
5. Email reminder
6. Flow of emails

After flow creation: A Record-Triggered Flow is created in Salesforce to automate business processes when a record is created or updated. In the WhatsNext Vision Motors project, the Flow is used to automate tasks such as dealer assignment, order processing, stock validation.



or

Test your Flow :



Test your Flow :

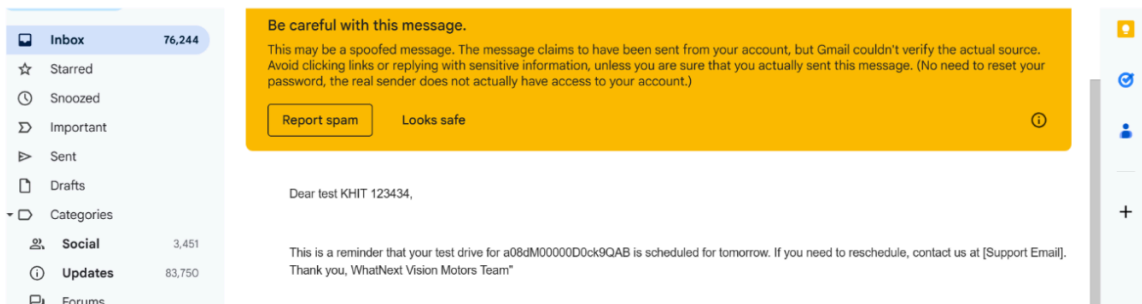


Figure: Automated Test Drive Reminder Notification

08:39



Reminder:Your Test Drive is
Tommorrow

Spam



SANJAY... Yesterday

to me



Why is this message in Spam?

This message is similar to messages that were identified as spam in the past.

Report as not spam



Dear Usershafa,This is Reminder that our test drive
a05jV000002WZxZQAW is tommorrow.if you need
to reschedule please contact us at
support@gmail.com

Thank you

← Reply

→ Forward

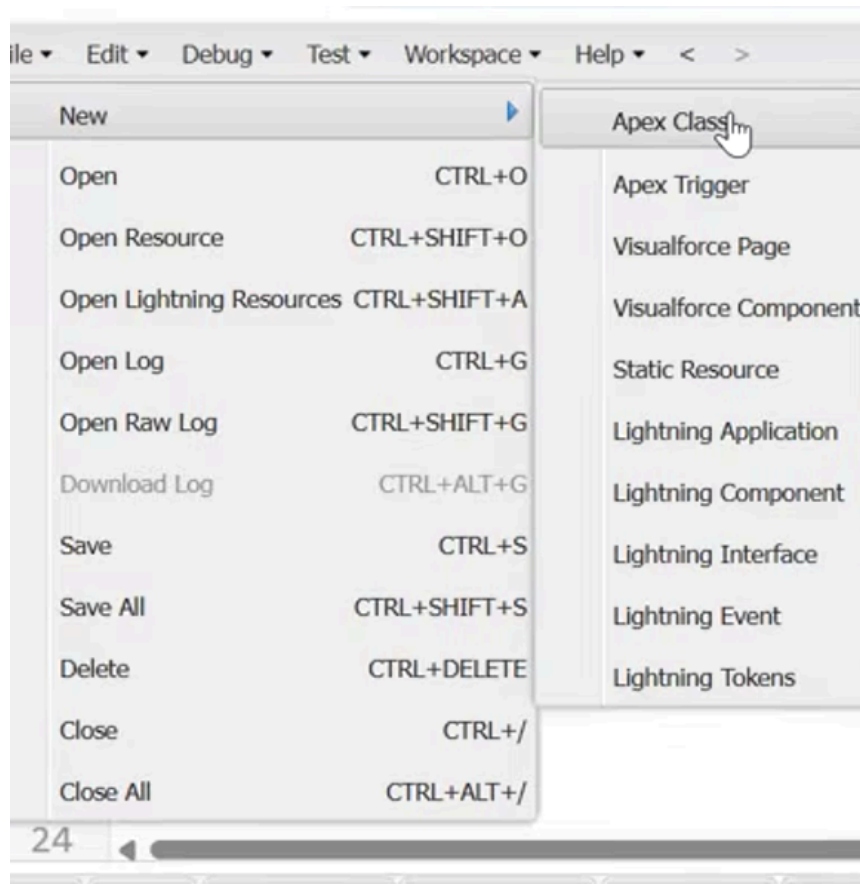


Milestone 7:

Create Apex and Trigger Batch Jobs :

Step 1 : Click Developer Console From Gear icon

Step 2 : click File then Select New then Select Apex.



Step 3 : Give Name For Apex Class

Class Name : VehicleOrderTriggerhandler

Step 4 : Write Apex code

```

File • Edit • Debug • Test • Workspace • Help • < >
OrderTriggerHandler.apxc | OrderTrigger.apxt | InventoryBatchJob.apxc | VehicleOrderTriggerHandler.apxc | VehicleOrderTrigger.apxt
Code Coverage: None | API Version: 52
1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean
4     if (isBefore) {
5         if (isInsert || isUpdate) {
6             preventOrderIfOutOfStock(newOrders);
7         }
8     }
9
10    if (isAfter) {
11        if (isInsert || isUpdate) {
12            updateStockOnOrderPlacement(newOrders);
13        }
14    }
15 }
16
17 // Method to prevent orders when the vehicle is out of stock
18 private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
19     Set<Id> vehicleIds = new Set<Id>();
20     for (Vehicle_Order__c order : orders) {
21         if (order.Vehicle__c != null) {
22             vehicleIds.add(order.Vehicle__c);
23         }
24     }
25
26     if (!vehicleIds.isEmpty()) {
27         Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
28         for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
29             vehicleStockMap.put(vehicle.Id, vehicle);
30     }

```

```

30     }
31 }
32
33 for (Vehicle_Order__c order : orders) {
34     if (vehicleStockMap.containsKey(order.Vehicle__c)) {
35         Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
36         if (vehicle.Stock_Quantity__c <= 0) {
37             order.addError('This vehicle is out of stock. Order cannot be placed.');

```

```

56     vehicleStockMap.put(vehicle.Id, vehicle);
57 }
58
59 List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
60 for (Vehicle_Order__c order : orders) {
61     if (vehicleStockMap.containsKey(order.Vehicle__c)) {
62         Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
63         if (vehicle.Stock_Quantity__c > 0) {
64             vehicle.Stock_Quantity__c -= 1;
65             vehiclesToUpdate.add(vehicle);
66         }
67     }
68 }
69
70 if (vehiclesToUpdate.isEmpty()) {
71     update vehiclesToUpdate;
72 }
73 }
74 }
75 }

```

Source Code :

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders,
    Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter,
    Boolean isInsert, Boolean isUpdate) {
        if (isBefore) {
            if (isInsert || isUpdate) {
                preventOrderIfOutOfStock(newOrders);
            }
        }

        if (isAfter) {
            if (isInsert || isUpdate) {
                updateStockOnOrderPlacement(newOrders);
            }
        }
    }

    // Method to prevent orders when the vehicle is out of stock
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c>
orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }

        if (!vehicleIds.isEmpty()) {
```

```

        Map<Id, Vehicle__c> vehicleStockMap = new Map<Id,
Vehicle__c>();
        for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM
Vehicle__c WHERE Id IN :vehicleIds]) {
            vehicleStockMap.put(vehicle.Id, vehicle);
        }

        for (Vehicle_Order__c order : orders) {
            if (vehicleStockMap.containsKey(order.Vehicle__c)) {
                Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
                if (vehicle.Stock_Quantity__c <= 0) {
                    order.addError('This vehicle is out of stock. Order cannot be
placed.');
```

```

                }
            }
        }
    }
}

// Method to update vehicle stock when an order is placed
private static void
updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
```

```

    for (Vehicle_Order__c order : orders) {
        if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
            vehicleIds.add(order.Vehicle__c);
        }
        Set<Id> vehicleIds = new Set<Id>();
    }
}
```

```

    if (!vehicleIds.isEmpty()) {
```

```

    Map<Id, Vehicle__c> vehicleStockMap = new Map<Id,
Vehicle__c>();
    for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM
Vehicle__c WHERE Id IN :vehicleIds]) {
        vehicleStockMap.put(vehicle.Id, vehicle);
    }

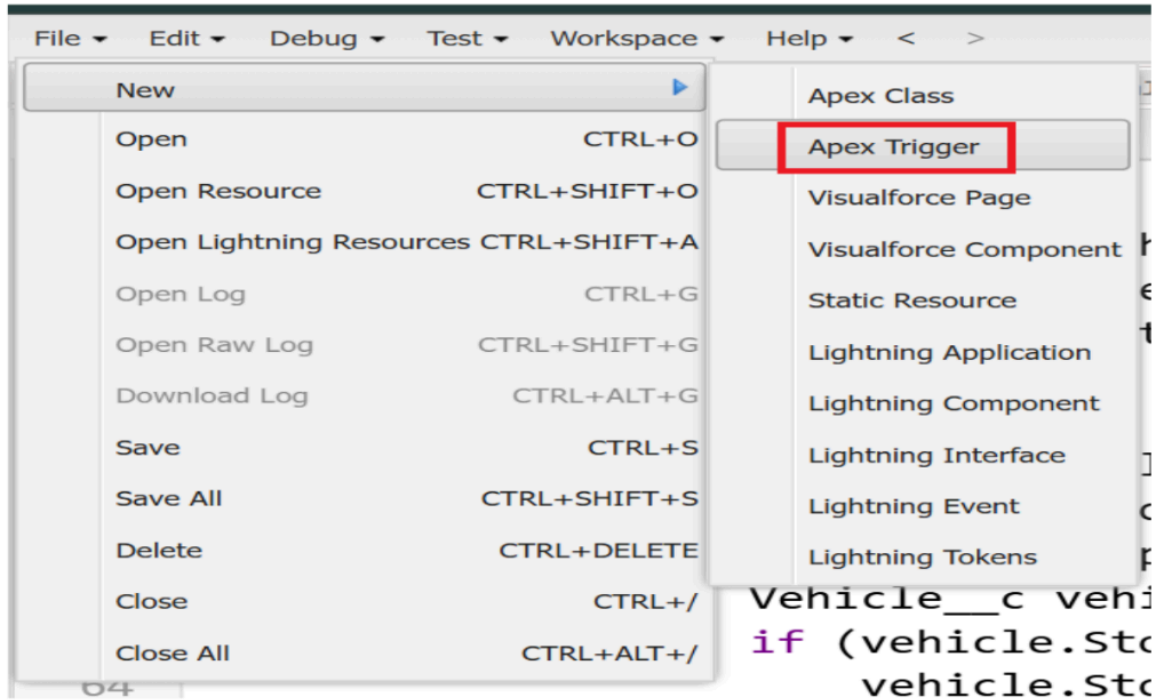
    List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
    for (Vehicle_Order__c order : orders) {
        if (vehicleStockMap.containsKey(order.Vehicle__c)) {
            Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
            if (vehicle.Stock_Quantity__c > 0) {
                vehicle.Stock_Quantity__c -= 1;
                vehiclesToUpdate.add(vehicle);
            }
        }
    }

    if (!vehiclesToUpdate.isEmpty()) {
        update vehiclesToUpdate;
    }
}

```

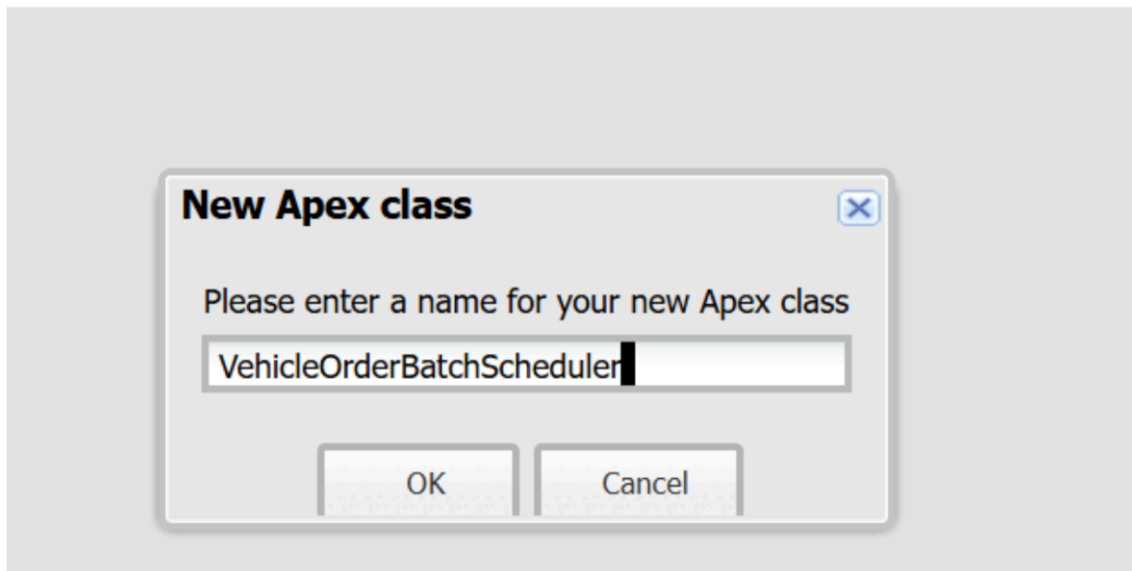
The screenshot shows a web form titled "Vehicle Order" with the ID "VC-0003". The form includes fields for "Vehicle Customer" (test KHIT 123434), "Vehicle" (1erwe), "Order Date" (21/02/2025), and "Status" (Pending). The "Owner" is listed as "Abdul Trainer". At the bottom, it shows "Created By" and "Created By" as "Abdul Trainer, 13/02/2025, 5:34 am". A red error message box is displayed in the center, stating "We hit a snag." and "Review the errors on this page." with a bullet point: "This vehicle is out of stock. Order cannot be placed." Below the error message are "Cancel" and "Save" buttons.

Step 5 : Write Trigger Handler



Step 6 : Writer Trigger Class Name and Select Vehicle Order Object

Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class



Step 7 : Call Apex Class in Trigger Class

Step 8 : Create Batch Job

A customer places an order, but the vehicle is out of stock.

- The order remains pending.



```
OrderTriggerHandler.apxc | OrderTrigger.apxt | InventoryBatchJob.apxc | VehicleOrderTriggerHandler.apxc | VehicleOrderTrigger.apxt
Code Coverage: None | API Version: 52
1 public class VehicleOrderTriggerHandler {
2
3     public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders, Boolean isBefore, Boolean isAfter, Boolean isOnDelete) {
4         if (isBefore) {
5             if (isInsert || isUpdate) {
6                 preventOrderIfOutOfStock(newOrders);
7             }
8         }
9
10        if (isAfter) {
11            if (isInsert || isUpdate) {
12                updateStockOnOrderPlacement(newOrders);
13            }
14        }
15    }
16
17    // Method to prevent orders when the vehicle is out of stock
18    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
19        Set<Id> vehicleIds = new Set<Id>();
20        for (Vehicle_Order__c order : orders) {
21            if (order.Vehicle__c != null) {
22                vehicleIds.add(order.Vehicle__c);
23            }
24        }
25
26        if (!vehicleIds.isEmpty()) {
27            Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
```

- After new stock is added, the batch job updates the order to confirmed.

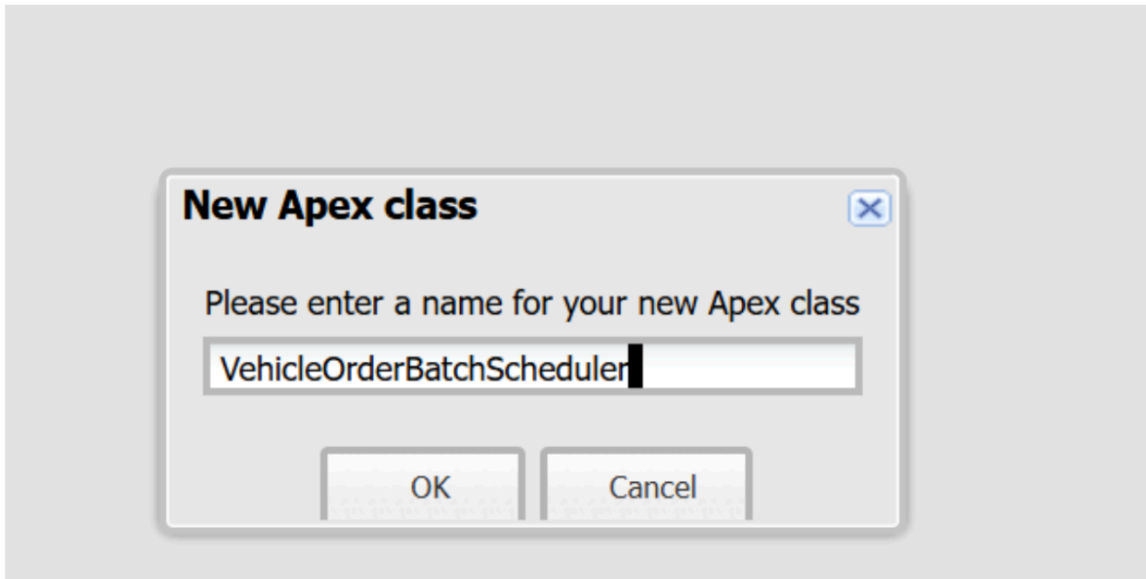
```
30     if (vehicleStockMap.containsKey(order.Vehicle__c)) {
31         Vehicle__c vehicle = vehicleStockMap.get(order.Vehicle__c);
32         if (vehicle.Stock_Quantity__c > 0) {
33             order.Status__c = 'Confirmed';
34             vehicle.Stock_Quantity__c -= 1;
35             ordersToUpdate.add(order);
36             vehiclesToUpdate.add(vehicle);
37         }
38     }
39 }
40
41 if (!ordersToUpdate.isEmpty()) {
42     update ordersToUpdate;
43 }
44
45 if (!vehiclesToUpdate.isEmpty()) {
46     update vehiclesToUpdate;
47 }
48 }
49 }
50
51 global void finish(Database.BatchableContext bc) {
52     System.debug('Vehicle order batch job completed.');
```

Code Coverage: None ▾

API Version: 62 ▾

```
1 ▾ global class VehicleOrderBatch implements Database.Batchable<sObject> {
2
3 ▾     global Database.QueryLocator start(Database.BatchableContext bc) {
4 ▾         return Database.getQueryLocator([
5             SELECT Id, Status__c, Vehicle__c
6             FROM Vehicle_Order__c
7             WHERE Status__c = 'Pending'
8         ]);
9     }
10
11 ▾     global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
12         Set<Id> vehicleIds = new Set<Id>();
13
14 ▾         for (Vehicle_Order__c order : orderList) {
15 ▾             if (order.Vehicle__c != null) {
16                 vehicleIds.add(order.Vehicle__c);
17             }
18         }
19
20 ▾         if (!vehicleIds.isEmpty()) {
21             Map<Id, Vehicle__c> vehicleStockMap = new Map<Id, Vehicle__c>();
22 ▾             for (Vehicle__c vehicle : [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]) {
23                 vehicleStockMap.put(vehicle.Id, vehicle);
24             }
25
26             List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
27             List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
28
29 ▾             for (Vehicle_Order__c order : orderList) {
30 ▾                 if (vehicleStockMap.containsKey(order.Vehicle__c)) {
```

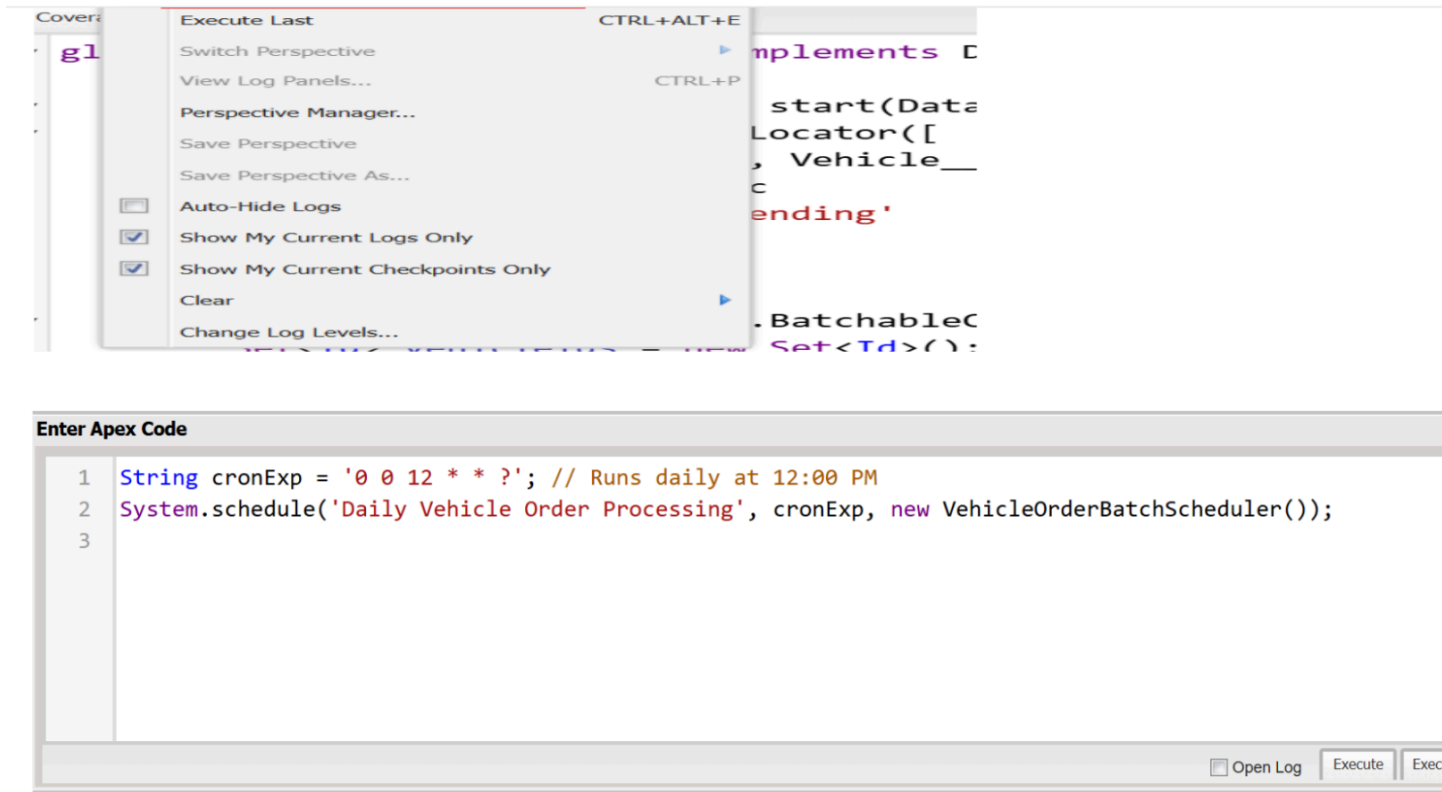
Step 9 : Create Schedule Class then revoked Batch Class in Schedule Class



Source Code :

```
global class VehicleOrderBatchScheduler implements Schedulable {  
    global void execute(SchedulableContext sc) {  
        VehicleOrderBatch batchJob = new VehicleOrderBatch();  
        Database.executeBatch(batchJob, 50); // 50 is the batch size  
    }  
}
```

Step 10 : **Schedule the Batch Job** To run the batch job **every night at midnight** :



Source Code :

```
String cronExp = '0 0 12 * * ?'; // Runs daily at 12:00 PM
```

```
System.schedule('Daily Vehicle Order Processing', cronExp, new  
VehicleOrderBatchScheduler());
```

```
System.schedule('Daily Vehicle Order Processing', cronExp, new  
VehicleOrderBatchScheduler());
```

Step 11 : You Can Check where this Schedule Job is Running

Scheduled jobs

Developer Edition

Search Setup

SetupHomeObject Manager

jobs

EnvironmentsJobsApex Flex QueueApex JobsBackground JobsBulk Data Load JobsScheduled Jobs

Didn't find what you're looking for? Try using Global Search.

SETUP

Scheduled Jobs

Percentage of Scheduled Jobs Used: 1%
You have currently used 1 scheduled Apex jobs out of an allowed organization limit of 100 active or scheduled jobs. To learn about how this limit is calculated and what contributes to it see the Lightning Platform Apex Limits topic.

View: All Scheduled Jobs Create New View

A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | Other

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger
Manage Del Pause Job	Daily Vehicle Order Processing	Bhargavi, Raguluru	9/10/2026, 9:32 AM		9/10/2026, 12:00 PM	Scheduled Apex	08efj00000e
Del	Metalytics Data Loader Job for Org : 00Dfj00000fFCR4	User Integration	9/4/2026, 11:58 AM	9/9/2026, 8:50 PM	9/10/2026, 8:50 PM	Autonomous Data Loader Job	08efj00000h
	Program Milestone Computation Cron Job	Process, Automated	9/4/2026, 11:57 AM	9/10/2026, 7:00 AM	9/10/2026, 11:59 AM	Program Milestone Computation Cron Job	08efj00000h
	Program Status Update Cron Job	Process, Automated	9/4/2026, 11:57 AM	9/10/2026, 5:01 AM	9/10/2026, 8:00 PM	Program Status Update Cron Job	08efj00000h

Developer Edition

Search Setup

SetupHomeObject Manager

jobs

EnvironmentsJobsApex Flex QueueApex JobsBackground JobsBulk Data Load JobsScheduled Jobs

Didn't find what you're looking for? Try using Global Search.

SETUP

Apex Jobs

Click here to go to the new batch jobs page

Apex Jobs

Monitor the status of all Apex jobs, and optionally, abort jobs that are in progress.

Percent of Asynchronous Apex Used: 0%
You have currently used 0 asynchronous Apex operations out of an allowed 24-hour organization limit of 250,000. To learn about how this limit is calculated and what contributes to the Lightning Platform Apex Limits topic.

View: All Create New View

Action	Submitted Date	Job Type	Status	Status Detail	Total Batches	Batches Processed	Failures	Submitted By	Completion Date	Apex Class	Apex Method
	9/10/2026, 9:32 AM	Scheduled Apex	Queued		0	0	0	Bhargavi, Raguluru		VehicleOrderBatchScheduler	
	9/10/2026, 4:26 AM	Scheduled Apex	Aborted		0	0	0	Bhargavi, Raguluru		VehicleOrderBatchScheduler	

ADVANTAGES:

- Centralized vehicle, dealer, customer, and order management.
- Automatic assignment of the nearest dealer.
- Prevents orders for out-of-stock vehicles.
- Improves vehicle stock management.
- Automates order status updates.
- Automated test drive reminder notifications.
- Reduces manual work and processing time.
- Improves customer satisfaction.
- Supports efficient order processing.

DISADVANTAGES:

- Salesforce configuration requires maintenance.
- Apex and automation require technical knowledge.
- Stock information must be kept accurate.
- Automation requires proper testing.
- Changes in configuration may affect dependent processes.

FUTURE SCOPE :

- Add AI-based vehicle recommendations.
- Integrate GPS-based dealer location services.
- Add online payment integration.
- Develop a mobile application.
- Add advanced sales and inventory dashboards.
- Integrate external inventory management systems.
- Add customer self-service features.
- Implement predictive vehicle demand analysis.

CONCLUSION :

The WhatsNext Vision Motors Salesforce CRM solution brings vehicle, dealer, customer, and order management into a centralized platform. The system automates stock validation, dealer assignment, order processing, test drive reminders, and stock updates using Salesforce Flows, Apex, Batch Apex, and Scheduled Apex. This improves operational efficiency, reduces manual effort, and provides a better customer experience.

APPENDIX

Main Salesforce Components :

Component	Purpose
Vehicle__c	— Stores vehicle details and stock information.
Dealer__c	— Stores dealer information and location details.
Customer__c	— Stores customer information.
Vehicle_Order__c	— Stores customer vehicle orders.
Test_Drive__c	— Manages scheduled test drives.
Service_Request__c	— Stores customer service requests.
Record-Triggered Flow	— Automates order and dealer processes.
Apex Trigger	— Validates stock and business rules.
Trigger Handler	— Manages reusable trigger logic.
Batch Apex	— Periodically processes vehicle stock.
Scheduled Apex	— Runs automated jobs at scheduled times.

Team Implementation Checklist :

Milestone

Completion Check

Milestone 1

Objects, fields, relationships, tabs, and Lightning App configured.

Milestone 2

Flows, stock validation, dealer assignment, and email automation completed.

Milestone 3

Apex Trigger, Trigger Handler, and business logic completed.

Milestone 4

Batch Apex and Scheduled Apex implemented and tested.

Final :

Functional testing and project screenshots collected.

Team Members

Name	Role
1.SANJAYA B	Team Leader
2.FATHIMA H	Member
3.MUBHARAK C	Member
4.ASHIK AHAMED A	Member

THANK YOU

